NETWORK AUTOMATION WITH PYTHON

A QUICK GUIDE TO AUTOMATING YOUR NETWORK

```
hostname = "netopsautomation.com"
output = net_connect(hostname).send_command
(show ip inter br")
print(output)
```

BY LAHCEN KHOUCHANE

netopsautomation.com

NETWORK AUTOMATION WITH PYTHON

A QUICK GUIDE TO AUTOMATING YOUR NETWORK

What is automation 3
Why Network Automation 3
Types of Network Automation 3
The Process of Automating Network Tasks 4
Why Python for Network Automation 5
Getting Started with Python 6
Python Basics 6
Variables in Python 7
Python Data Types 7
IF Statement (Conditional Execution) 15
Loop Statements (for & while) 16
Functions 18
Error Handling 19
I/O Operations 19
Working with CSV Files 20
Working with JSON 22
Working with XML 23
ipaddress Module 25
netaddr Library 26
os Module 28
PrettyTable Library 30
Regular Expressions (re Module) 31
Command-Line Arguments 34
python-dotenv Library 35
Working with Sockets in Python 36
Lab Preparation: Setting Up the CSR1000V Router in GNS3 38
Telnet (telnetlib) 39
SSH and Paramiko 41
Netmiko 43
Screen Scraping 46
Jinja 52
NETCONF and YANG 54
ncclient: Python Library for NETCONF 55
Logging 57
Pandas for Network Data Analysis 57
Threading: Speeding Up Network Automation 59
Config Diff: Comparing Network Configurations 59
Conclusion 60

What is automation

Network automation refers to the use of software and tools to handle network tasks with minimal manual input. In our line of work, this means moving away from the traditional, hands-on approach like logging into routers, switches, or firewalls to make configuration changes or gather data and letting automation tools take over those repetitive tasks.

By using scripts, APIs, and orchestration platforms, we can automate network provisioning, configuration management, monitoring, and even troubleshooting. This not only speeds up operations but also reduces the chance of human error and allows us to focus on more strategic, high-level tasks.

In short, network automation helps streamline day-to-day operations, lowers dependency on manual effort, and improves the overall efficiency and reliability of the network.

Why Network Automation

Network automation brings several key benefits that directly impact day-to-day operations and long-term network stability:

- **Reduced Errors:** Manual configurations are one of the leading causes of network outages some estimates say up to 80%. Automation minimizes human error by using tested, repeatable scripts and workflows.
- **Standardized Processes:** With automation, configurations and policies are applied consistently across all devices. This standardization simplifies management, accelerates deployments, and makes scaling the network much more manageable.
- **Lower Risk, Higher Reliability:** Automated workflows reduce the risk associated with configuration drift and inconsistent device setups. By ensuring repeatable and predictable operations, network automation enhances overall reliability and reduces the time spent troubleshooting.

Types of Network Automation

Network automation can take many forms depending on what tasks you're looking to streamline. Here are a few common types that bring real value to network operations:

- **Device Provisioning:** This involves automating the initial setup and configuration of network devices. For example, instead of manually setting up each router or switch, you can generate and apply a predefined configuration template to new devices during deployment.
- **Data Collection:** Automation can be used to run commands (like show commands) across multiple devices, collect the output, and store it for further analysis. This is especially useful for performance monitoring, inventory management, or identifying potential issues across the network.

- **Automated Reporting:** Once you're collecting data programmatically, the next step is to turn that data into useful insights. With automation, you can generate custom reports such as device health, interface usage, or configuration compliance that update dynamically and help guide decision-making.

The Process of Automating Network Tasks

Successfully automating tasks in a network environment requires a structured and strategic approach. Here's a breakdown of the key steps involved:

1. **Define Objectives:**

   Start by identifying which tasks you want to automate whether it’s device provisioning, configuration management, monitoring, or troubleshooting. Clear goals will guide tool selection and workflow design.

2. **Choose Automation Tools:**

   Select tools that align with your technical environment and objectives. Common tools include:

   a. Ansible for configuration management and orchestration
   b. Python with libraries like Netmiko, NAPALM, Paramiko, and ncclient for scripting and device interaction
   c. Vendor-specific platforms like Cisco DNA Center or Juniper PyEZ for integrated solutions

3. **Device Inventory Management:**

   Maintain an up-to-date inventory of all network devices. This inventory should include IP addresses, device types, roles, credentials, and OS/platform details. It can be stored in a flat file (e.g., YAML, JSON) or in a centralized database.

4. **Task Orchestration:**

   Develop structured workflows to automate multi-step tasks. For example, you might script a sequence that creates a new VLAN, applies the config to all access switches, and verifies success with post-checks.

5. **Error Handling and Logging:**

   Implement mechanisms to catch and log errors. This ensures any issues during execution are captured, making it easier to debug and refine your automation scripts or playbooks.

6. **Security and Compliance:**

   Secure your automation framework by encrypting sensitive data (e.g., passwords, keys), using role-based access controls, and ensuring that automation aligns with compliance policies.

7. **Testing and Validation:**

   Validate your automation workflows in a test or staging environment before pushing changes to production. Use unit tests or dry runs to confirm outcomes and prevent misconfiguration.

8. **Version Control:**

   Store all automation scripts, configs, and templates in a version control system like Git. This promotes collaboration, enables rollbacks, and tracks historical changes.

Why Python for Network Automation

Python has become the go-to language for network automation, and for good reason. Here's why it's a favorite among network engineers:

1. **Easy to Learn and Use:**

   Python's clean, human-readable syntax makes it easy for network engineers—even those without a traditional programming background—to pick up and start writing useful automation scripts quickly. You don't need to be a developer to get value from Python.

2. **Powerful Library Ecosystem for Networking:**

   Python offers a wide range of libraries purpose-built for network automation, which significantly reduce the need to "build from scratch." Popular libraries include:

   a. **Netmiko:** Simplifies SSH connections to network devices.
   b. **NAPALM:** Provides a consistent API for multivendor device interaction.
   c. **Nornir:** A pure Python automation framework for orchestrating complex workflows.
   d. **Paramiko, ncclient:** Useful for SSH and NETCONF-based automation.

3. **Data Handling, Analysis, and Visualization:**

   Python isn't just about sending configs. It also excels at collecting, parsing, and analyzing network data. With libraries like Pandas, Matplotlib, or Seaborn, you can turn raw CLI output or SNMP data into actionable insights and visual reports. This is especially helpful for spotting trends, anomalies, or performance bottlenecks.

4. **Integration with Other Tools:**

   Python plays well with other tools and platforms, including REST APIs, monitoring systems (like Prometheus), ticketing tools (like ServiceNow), and even CI/CD pipelines. This makes it a flexible language for full-stack network automation.

Getting Started with Python

Before diving into network automation, let's set up your Python environment and write your very first script.

- **Step 1: Install Python 3**

  To begin, you'll need Python 3.12 installed on your computer. Download it from the official site: https://www.python.org/downloads/

- **Step 2: Choose a Text Editor or IDE**

  You can write Python code using any text editor you like, but here are two popular options:

  a. **Visual Studio Code (VS Code):** Lightweight and widely used, great for beginners.
  b. **PyCharm:** A full-featured IDE built specifically for Python.

- **Step 3: Write Your First Script**

  Let's start with something simple. Create a new folder and name it: ***python_basics***

  Inside that folder, create a new file and name it: ***hello.py***

  Open ***hello.py*** in your editor and type the following line of code:

```
print("Hello World!")
```

- **Step 4: Run the Script**
  - Open **Command Prompt (CMD)** or **PowerShell**
  - Navigate to the ***python_basics folder*** using the cd command.
  - Run your script with:

```
python hello.py
```

Output:

```
Hello, World!
```

Voilà! You've just written and executed your first Python script.

Python Basics

Before jumping into automation tasks, it's important to understand the building blocks of Python. This section covers the fundamentals that you'll use in nearly every script.

Variables in Python

Python uses **dynamic typing**, meaning you don't need to declare a variable's type before assigning a value. A variable simply points to a value in memory.

Variable Naming Rules:

- Use letters, digits, and underscores (_) only.
- Must **not start with a digit**.
- Special characters like **@, $, %** are not allowed.
- Python is **case-sensitive**, so ***MyVar*** and ***myvar*** are different.

Example:

```
a = 1
b = "hello"
print(a, b)
```

Output:

```
1 hello
```

Python Data Types

Python supports several built-in data types, each useful for different kinds of data and operations:

- **Numbers:**
  - **int:** Stores whole numbers (positive, negative, or zero).
  - **float:** Stores numbers with decimals.
- **Strings (str):**

  Represent text data enclosed in single (') or double (") quotes. Can also use triple quotes (''' or """) for multi-line strings.

- **Lists (list):**

  Ordered collections of items, written within square brackets **[]**. Elements can be of different data types.

  Mutable: You can add, remove, or change elements after creation.

- **Tuples (tuple):**

  Similar to lists but ordered and immutable (cannot be changed after creation). Written within parentheses **()**.

- **Dictionaries (dict):**

  Unordered collections of key-value pairs. Keys must be unique and immutable (often strings). Values can be any data type. Written within curly braces **{}**.

- **Sets (set):**

  Unordered collections of unique elements. Useful for checking membership or removing duplicates. Written within curly braces **{}**.

Showcase various data types in Python:

```
# Numerics (int, float)
num_int = 42
num_float = 3.14
# Strings
name = "Python"
greeting = 'Hello, world!'
# Lists
network_vendors = ["cisco", "nokie", "huawei", "juniper", "zte"]
numbers = [1, 2, 3, 4, 5]
# Tuples (ordered, immutable collection of items)
weekdays = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
# Dictionaries
router_info = {"name": "R1",
               "ipaddress": "192.168.1.1",
               "technology": "cisco",
               "module": "csr1000v"}
# Sets
unique_ipaddress = {"192.168.1.1", "192.168.1.2", "192.168.1.3", "192.168.1.4"}
# Print the data types of each variable
print("Integer:", type(num_int))
print("Float:", type(num_float))
print("String:", type(name))
print("List:", type(network_vendors))
print("Tuple:", type(weekdays))
print("Dictionary:", type(router_info))
print("Set:", type(unique_ipaddress))
```

Output:

```
Integer: <class 'int'>
Float: <class 'float'>
String: <class 'str'>
List: <class 'list'>
Tuple: <class 'tuple'>
Dictionary: <class 'dict'>
Set: <class 'set'>
```

Operations on Strings

Strings are one of the most commonly used data types in Python, especially when dealing with device names, log messages, and CLI output. Python offers a range of built-in operations that make working with strings efficient and flexible.

1. **Concatenation:**

Example:

```
str1 = "Hello"
str2 = "World"
str3 = str1 + " " + str2
print(str3)
```

Output:

```
Hello World
```

2. **Indexing and Slicing:**

   Strings in Python behave like ordered sequences of characters, meaning you can access and manipulate individual characters or subsets of characters using indexing and slicing.

Example:

```
str = "Python"
# indexing
first_char = str[0]
print("Indexing: ",first_char)
# Slicing
substring = str[1:4]
print("Slicing: ",substring)
```

Output:

```
Indexing: P
Slicing: yth
```

3. **Escape Sequences:**

   Escape sequences are special characters in Python strings that begin with a backslash (\). They are used to represent characters that are either non-printable or would otherwise be difficult to include directly in a string, like quotes or newlines.

Escape Sequence	Description
\"	Inserts a double quote
\'	Inserts a single quote
****	Inserts a backslash
\n	New line
\t	Tab (horizontal spacing)

Example:

```
str = "This is a string with a \"double quote\"."
print(str)
```

Output:

```
This is a string with a "double quote".
```

4. **String Methods:**

   Python provides a rich set of built-in methods for string manipulation. Here are some commonly used methods:

   - **strip():** Removes leading and trailing whitespaces from the string.
   - **lower():** Converts the string to lowercase.
   - **upper():** Converts the string to uppercase.
   - **replace(old, new, count):** Replaces all occurrences of a substring (old) with another substring (new) within the string. Optionally, you can specify the number of replacements (count).
   - **split(sep):** Splits the string into a list of substrings based on the specified separator (sep). By default, it splits on whitespace.
   - **find(sub):** Finds the index of the first occurrence of a substring (sub) within the string. It returns -1 if the substring is not found.
   - **startswith(sub):** Checks if the string starts with the given substring (sub). Returns True or False.
   - **endswith(sub):** Checks if the string ends with the given substring (sub). Returns True or False.

Example:

```
str = "   CSR1000v is cisco router, IPaddress 10.15.1.1"
trimmed_str = str.strip()
print("Remove leading and trailing whitespaces:\n", trimmed_str)

lower_str = str.lower()
print("Convert to lowercase:\n",lower_str)

upper_str = str.upper()
print("Convert to uppercase:\n",upper_str)

replaced_str = str.replace("10.15.1.1", "192.168.1.10")
print("Replace substring:\n",replaced_str)

split_str = str.split(",")
print("split substring:\n",split_str)


index = str.find("cisco")
print("Find the index of the 'cisco': ", index)

print("startswith:", str.startswith("csr1000v"))
print("endswith:",str.endswith("10.15.1.1"))
```

Output:

```
Remove leading and trailing whitespaces:
 CSR1000v is cisco router, IPaddress 10.15.1.1

Convert to lowercase:
    csr1000v is cisco router, ipaddress 10.15.1.1

Convert to uppercase:
    CSR1000V IS CISCO ROUTER, IPADDRESS 10.15.1.1

Replace substring:
    CSR1000v is cisco router, IPaddress 192.168.1.10

split substring:
 ['   CSR1000v is cisco router', ' IPaddress 10.15.1.1']

Find the index of the 'cisco':  15
startswith: False
endswith: True
```

5. **String Formatting:**

   String formatting allows you to insert variables or expressions directly into a string, making your output dynamic and readable. This is especially useful in automation scripts for generating messages, logs, or configurations. **F-strings** are the most modern and readable way to embed variables into strings. Just prefix the string with f and wrap variables or expressions in {}.

Example:

```
router_name = "csr1000v"
username = "user"
greeting = f"Welcome {username} your are connected to {router_name}.\nAuthorized Access Only"
print(greeting)
```

Output:

```
Welcome user your are connected to csr1000v.
Authorized Access Only
```

Operations on Lists

Lists are a versatile data type in Python, especially useful in network automation for storing device names, IP addresses, logs, or command outputs. Python provides powerful built-in methods to manipulate lists easily.

1. **Adding Elements**

You can add elements to a list using:

- **append(element)**: Adds an element at the end.
- **insert(index, element):** Inserts an element at a specific index, shifting the rest to the right.

Example:

```
ip_addresses = ["192.168.1.1", "10.0.0.1", "8.8.8.8"]
ip_addresses.append("172.16.1.5")
ip_addresses.insert(1, "11.11.11.11")
print(ip_addresses)
```

Output:

```
['192.168.1.1', '11.11.11.11', '10.0.0.1', '8.8.8.8', '172.16.1.5']
```

2. **Removing Elements**

You can remove elements in two common ways:

- **remove(element):** Deletes the first matching element. (Raises an error if not found.)
- **pop(index):** Removes and returns the element at the given index. Defaults to the last element if no index is provided**.**

Example:

```
ip_addresses = ["192.168.1.1", "10.0.0.1", "8.8.8.8", "csr1000v", "R1"]
ip_addresses.remove("R1")
ip_addresses.pop(3)
print(ip_addresses)
```

Output:

```
['192.168.1.1', '10.0.0.1', '8.8.8.8']
```

3. **Getting the Length of a List**

Use **len()** to find out how many elements are in a list.

Example:

```
ip_addresses = ["192.168.1.1", "10.0.0.1", "8.8.8.8"]
print("Length of the list is:", len(ip_addresses))
```

Output:

```
Length of the list is: 3
```

Operations on Dictionaries

Dictionaries in Python are highly efficient data structures used to store key-value pairs. They're perfect for modeling structured data like interface configurations, device attributes, or command outputs—making them especially useful in network automation.

1. **Accessing Values :** Use the key in square brackets **[]** to retrieve a value.

Example:

```
interface_info = {
    "interface_name": "GigabitEthernet1/0/1",
    "ip_address": "192.168.1.1",
    "subnet_mask": "255.255.255.0",
    "description": "Network Automation With Python",
    "enabled": True,
    "negotiation_auto": False,
}
print(f"Interface Name  : {interface_info['interface_name']}")
print(f"IP Address      : {interface_info['ip_address']}")
print(f"Subnet Mask     : {interface_info['subnet_mask']}")
print(f"Description     : {interface_info['description']}")
print(f"Interface State : {interface_info['enabled']}")
print(f"Negotiation Auto: {interface_info['negotiation_auto']}")
```

Output :

```
interface name      : GigabitEthernet1/0/1
IPAddress           : 192.168.1.1
Mask                : 255.255.255.0
Description         : Network Automation With Python
Interface state     : True
Negotiation auto    : False
```

2. **Adding Key-Value Pairs:** Use square brackets with a new key to add a new pair.

Example:

```
from pprint import pprint
interface_info= {
    "interface_name": "GigabitEthernet1/0/1",
    "ip_address": "192.168.1.1",
    "subnet_mask": "255.255.255.0",
    "description": "Network Automation With Python",
    "enabled": True,
    "negotiation_auto": False,
}
# add new key to dict
interface_info["vlan"] = 100
pprint(interface_info)
```

Output :

```
{'description': 'Network Automation With Python',
 'enabled': True,
 'interface_name': 'GigabitEthernet1/0/1',
 'ip_address': '192.168.1.1',
 'negotiation_auto': False,
 'subnet_mask': '255.255.255.0',
 'vlan': 100}
```

3. **Removing Key-Value Pairs :** Use **pop(key)** to remove a key and its value.

Example:

```
from pprint import pprint
interface_info= {
    "interface_name": "GigabitEthernet1/0/1",
    "ip_address": "192.168.1.1",
    "subnet_mask": "255.255.255.0",
    "description": "Network Automation With Python",
    "enabled": True,
    "negotiation_auto": False,
}
interface_info.pop("negotiation_auto")
pprint(interface_info)
```

Output:

```
{'description': 'Network Automation With Python',
 'enabled': True,
 'interface_name': 'GigabitEthernet1/0/1',
 'ip_address': '192.168.1.1',
 'subnet_mask': '255.255.255.0'}
```

4. **Getting Keys, Values, and Items**
    - **keys():** Returns all keys
    - **values():** Returns all values
    - **items():** Returns key-value pairs as tuples

Example:

```
interface_info= {
    "interface_name": "GigabitEthernet1/0/1",
    "ip_address": "192.168.1.1",
    "subnet_mask": "255.255.255.0",
    "description": "Network Automation With Python",
    "enabled": True,
    "negotiation_auto": False,
}
```

```
print(interface_info.keys())
print(interface_info.values())
print(interface_info.items())
```

Output :

```
dict_keys(['interface_name', 'ip_address', 'subnet_mask', 'description', 'enabled', 'negotiation_auto'])

dict_values(['GigabitEthernet1/0/1', '192.168.1.1', '255.255.255.0', 'Network Automation With Python', True, False])

dict_items([('interface_name', 'GigabitEthernet1/0/1'), ('ip_address', '192.168.1.1'), ('subnet_mask', '255.255.255.0'), ('description', 'Network Automation With Python'), ('enabled', True), ('negotiation_auto', False)])
```

5. **Iterating Over a Dictionary :** Use a for loop with **.items()** to iterate through keys and values:

Example:

```
interface_info= {
    "interface_name": "GigabitEthernet1/0/1",
    "ip_address": "192.168.1.1",
    "subnet_mask": "255.255.255.0",
    "description": "Network Automation With Python",
    "enabled": True,
    "negotiation_auto": False,
}
for key, value in interface_info.items():
    print(f"{key}: {value}")
```

Output :

```
interface_name: GigabitEthernet1/0/1
ip_address: 192.168.1.1
subnet_mask: 255.255.255.0
description: Network Automation With Python
enabled: True
negotiation_auto: False
```

IF Statement (Conditional Execution)

The if statement in Python allows you to control the flow of your program by executing code only when certain conditions are met. This is essential for decision-making in scripts like validating user input, checking device states, or branching logic based on configuration data.

Example:

```
# Check interface status
interface_status = "up"

if interface_status == "up":
    print("Interface is operational.")
else:
    print("Interface is down. Please troubleshoot.")
```

Output :

```
Interface is operational.
```

Loop Statements (for & while)

Loops are essential in Python for performing repetitive tasks—a common scenario in network automation where you often need to check devices, iterate over interfaces, or retry connections.

1. **For Loop :**

   The for loop is ideal when you need to iterate over a known collection of items like IP addresses, hostnames, or commands.

Example:

```
# List of IP addresses
ip_addresses = ["192.168.1.1", "10.0.0.2", "8.8.8.8"]

for ip in ip_addresses:
    print(f"Connecting to IPAddress: {ip}")
```

Output:

```
Connecting to IPAddress: 192.168.1.1
Connecting to IPAddress: 10.0.0.2
Connecting to IPAddress: 8.8.8.8
```

2. **While Loop**

   The while loop runs as long as a condition remains true. It's useful when you don’t know ahead of time how many times you’ll need to loop like waiting for a connection or retrying a task until success.

Example:

```
# Wait for user input to establish a network connection
connected = False
while not connected:
  user_choice = input("Connect to network? (y/n): ")
  if user_choice.lower() == 'y':
```

```
    # Simulate network connection logic (replace with actual connection code)
    print("Connecting...")
    connected = True  # Connection successful (replace with actual success check)
    print("Connected!")
  else:
    print("Connection attempt cancelled.")
```

Output:

```
Connect to network? (y/n): n
Connection attempt cancelled.
Connect to network? (y/n): y
Connecting...
Connected!
```

Loop Control Statements (break, continue, pass)

In Python, loop control statements let you fine-tune how loops behave. These are especially useful when working with device lists, response validation, or retry logic in network automation scripts.

1. **break – Exit the Loop Early:** Use ***break*** to immediately exit a loop when a certain condition is met.

Example: Stop When Unreachable IP Is Found

```
ip_list = ["192.168.1.1", "10.0.0.2", "0.0.0.0"]  # Assume 0.0.0.0 is invalid

for ip in ip_list:
    if ip == "0.0.0.0":
        print("Invalid IP address found. Stopping the loop.")
        break
    print(f"Pinging {ip}...")
```

Output:

```
Pinging 192.168.1.1...

Pinging 10.0.0.2...

Invalid IP address found. Stopping the loop.
```

2. **continue – Skip Current Iteration :** Use ***continue*** to skip the rest of the current loop iteration and move on to the next.

Example: Skip Private IPs

```
ip_list = ["192.168.1.1", "8.8.8.8", "10.0.0.1"]

for ip in ip_list:
    if ip.startswith("192.168") or ip.startswith("10."):
        continue  # Skip internal IPs
    print(f"Sending traffic to {ip}")
```

Output :

```
Sending traffic to 8.8.8.8
```

3. **pass – Placeholder for Future Code :** ***pass*** does nothing. It's used when a statement is syntactically required but you don't want any action yet—often in scaffolding or testing.

Example: Placeholder for Device Check Logic

```
devices = ["router1", "switch1", "firewall1"]

for device in devices:
    if device == "firewall1":
        pass  # To be implemented later
    else:
        print(f"Checking {device} configuration...")
```

Output :

```
Checking router1 configuration...
Checking switch1 configuration...
```

Functions

A function is a reusable block of code that performs a specific task. It helps organize your scripts, avoid duplication, and break down complex logic into manageable parts an essential skill in any network automation workflow.

What is a Function?

- Defined using the **def** keyword.
- Takes parameters (optional) as input.
- Can **return** a value (optional).
- Can be reused multiple times with different inputs.

Example: Convert Domain Name to IP Address

This function uses Python's socket module to resolve a domain name into an IP address something you might use when verifying DNS resolution in scripts.

```
import socket

def get_ip_address(domain_name):
    ip_address = socket.gethostbyname(domain_name)
    return ip_address

domain = "cisco.com"
ip = get_ip_address(domain)
print(f"The IP address of {domain} is: {ip}")
```

Output :

```
The IP address of cisco.com is: 72.163.4.185
```

How It Works:

- **get_ip_address()** is the function name.
- It takes **domain_name** as a parameter.
- Inside, it calls **socket.gethostbyname()** to resolve the domain.
- The result is returned and stored in the variable **ip**.
- You can reuse this function to resolve any domain.

Error Handling

When working with network devices, APIs, or external systems, errors are bound to happen—especially with connectivity issues, invalid inputs, or timeouts. Python's **try-except** block lets you catch and handle these errors gracefully, rather than crashing your script.

What is try-except?

- The **try** block contains code that might raise an error
- If an error occurs, the **except** block runs instead—acting as a backup plan.
- This is especially useful in automation where stability is key.

Example: Error Handling for Domain Resolution

```
import socket

def get_ip_address(domain_name):
    try:
        ip_address = socket.gethostbyname(domain_name)
        print(f"The IP address of {domain_name} is: {ip_address}")
    except Exception as e:
        print(f"Error: {e}")

get_ip_address("cisco")  # Intentional typo (should be cisco.com)
```

Output :

```
Error: [Errno 11001] getaddrinfo failed
```

I/O Operations

Input/Output (I/O) operations allow your Python scripts to interact with users and read/write data to files—an essential part of automating tasks like logging device output, storing configurations, or receiving user input for dynamic actions.

1. **User Input :** Python's built-in ***input()*** function lets you pause the program to accept user input, which is always returned as a string.

Example: Basic User Prompt

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

Output :

```
What is your name? Python
Hello, Python!
```

2. **File Handling :** Python's ***open()*** function allows you to interact with files for reading, writing, or appending content.

   Common File Modes:

Mode	Description
"r"	Read (default). File must exist.
"w"	Write. Creates or overwrites file.
"a"	Append. Adds to the end of the file.

   a. **Reading from a File :** Let's read from a file named **data.txt**. Make sure the file exists and contains some content first.

Example:

```
with open("data.txt", "r") as file:
    contents = file.read()
    print("The content of the file is:\n\t", contents)
```

Output :

```
The content of the file is:
     Network Automation With Python!
```

   b. **Writing to a File :** This will create or overwrite **data.txt** with the specified content.

Example:

```
data = "Network Automation With Python!"
with open("data.txt", "w") as file:
    file.write(data)
```

> Using **with open(...)** ensures the file is automatically closed, which is best practice to prevent resource leaks.

Working with CSV Files

CSV files are a common file format that you will frequently encounter. You may use them to store various data, such as device information like router and switch names, their IP addresses, and more. Python's built-in **csv** module makes it easy to read from and write to CSV files, which is especially useful for device inventory management or report generation in network automation.

1. **Reading CSV Files :** Create a CSV file name it ***data.csv*** and write the following data into it.

```
device_name,type,ipaddress,location
R1,router,10.15.1.1,SALE-A
SW1,switch,10.15.2.1,SALE-A
SW2,switch,10.15.2.3,SALE-C
SW3,switch,10.15.2.1,SALE-B
```

Example 1: Reading CSV Rows as Lists

```
import csv

with open('data.csv', 'r') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
    print(row)
```

Output :

```
['device_name', 'type', 'ipaddress', 'location']
['R1', 'router', '10.15.1.1', 'SALE-A']
['SW1', 'switch', '10.15.2.1', 'SALE-A']
['SW2', 'switch', '10.15.2.3', 'SALE-C']
['SW3', 'switch', '10.15.2.1', 'SALE-B']
```

Example 2: Reading CSV Rows as Dictionaries

Using ***DictReader***, you can access each row as a dictionary, which is more readable and usable in automation scripts.

```
import csv

with open('data.csv', 'r') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
    print(row)
```

Output :

```
{'device_name': 'R1', 'type': 'router', 'ipaddress': '10.15.1.1', 'location': 'SALE-A'}
{'device_name': 'SW1', 'type': 'switch', 'ipaddress': '10.15.2.1', 'location': 'SALE-A'}
{'device_name': 'SW2', 'type': 'switch', 'ipaddress': '10.15.2.3', 'location': 'SALE-C'}
{'device_name': 'SW3', 'type': 'switch', 'ipaddress': '10.15.2.1', 'location': 'SALE-B'}
```

2. **Writing to CSV Files**

Example 1: Writing Rows as Lists

```
import csv

data = [
['device_name', 'type', 'ipaddress', 'location'],
```

```
['R1', 'router', '10.15.1.1', 'SALE-A'],
['SW1', 'switch', '10.15.2.1', 'SALE-A'],
['SW2', 'switch', '10.15.2.3', 'SALE-C'],
['SW3', 'switch', '10.15.2.1', 'SALE-B'],
]

with open('output.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(data)
```

Example 2: Writing Rows as Dictionaries

Using ***DictWriter*** provides better control and clarity when dealing with structured data.

```
import csv

data = [
 {'device_name': 'R1', 'type': 'router', 'ipaddress': '10.15.1.1', 'location': 'SALE-A'},
 {'device_name': 'SW1', 'type': 'switch', 'ipaddress': '10.15.2.1', 'location': 'SALE-A'},
]

with open('output2.csv', 'w', newline='') as csvfile:
    headers = ["device_name", "type", "ipaddress", "location"]
    writer = csv.DictWriter(csvfile, fieldnames=headers)
    writer.writeheader()
    writer.writerows(data)
```

Working with JSON

JSON (JavaScript Object Notation) acts as a common language for devices to exchange information. It's commonly used in REST APIs, network device configuration exports, and automation tools. Python provides a built-in module called **json** for working with JSON files.

1. **Reading JSON data:**
    - **json.load():** Reads JSON from a file and converts it to a Python object
    - **json.loads():** Reads JSON from a string and converts it to a Python object

Create a JSON file name it **data.json** and write the following data into it :

```
[
    {"device_name": "R1",
     "type": "router",
     "ipaddress": "10.15.1.1",
     "location": "SALE-A"},

    {"device_name": "SW1",
     "type": "switch",
     "ipaddress": "10.15.2.1",
     "location": "SALE-A"}
]
```

Example: Reading JSON from a File

```
import json
from pprint import pprint

with open("data.json", "r") as json_file:
    json_inventory = json_file.read()
inventory = json.loads(json_inventory)
pprint(inventory)
```

Output :

```
[{'device_name': 'R1',
  'ipaddress': '10.15.1.1',
  'location': 'SALE-A',
  'type': 'router'},
 {'device_name': 'SW1',
  'ipaddress': '10.15.2.1',
  'location': 'SALE-A',
  'type': 'switch'}]
```

2. **Writing JSON Data**
    - **json.dump():** Writes a Python object to a file in JSON format.
    - **json.dumps():** Converts a Python object into a JSON-formatted string.

Example: Writing to a JSON File

```
import json

inventory = [
    {'device_name': 'R1', 'ipaddress': '10.15.1.1', 'location': 'SALE-A', 'type': 'router'},
    {'device_name': 'SW1', 'ipaddress': '10.15.2.1', 'location': 'SALE-A', 'type': 'switch'}
]

with open("data.json", "w") as json_file:
    json.dump(inventory, json_file, indent=4)
```

Working with XML

XML (eXtensible Markup Language) is another widely used data format especially in network configuration exports, NETCONF, and legacy systems. Python's ***xml.etree.ElementTree*** module offers an intuitive way to parse and interact with XML files as tree structures.

Create a XML file name it **data.xml** and write the following data into it.

```
<?xml version="1.0" encoding="UTF-8"?>
<devices>
    <device>
        <device_name>R1</device_name>
        <type>router</type>
        <ipaddress>10.15.1.1</ipaddress>
```

```
    <port>22</port>
    <connection_type>ssh</connection_type>
    <location>SALE-A</location>
  </device>
  <device>
    <device_name>SW1</device_name>
    <type>switch</type>
    <ipaddress>10.15.2.1</ipaddress>
    <port>22</port>
    <connection_type>ssh</connection_type>
    <location>SALE-A</location>
  </device>
</devices>
```

1. **Reading XML Data Using ElementTree :**

   To work with XML, you typically:

   - Use **ElementTree.parse()** to load the file.
   - Use **.getroot()** to access the top-level element.
   - Traverse the XML structure using **.find()** or **.iter()**.

Example: Reading Device Data from an XML File

```
import xml.etree.ElementTree as ET

tree = ET.parse("data.xml")
root = tree.getroot()

for child in root.iter("device"):
    device_name = child.find("device_name").text
    device_type = child.find("type").text
    ipaddress = child.find("ipaddress").text
    port = child.find("port").text
    connection_type = child.find("connection_type").text
    location = child.find("location").text
    print("|-------------------------|")
    print(f"device_name     : {device_name}")
    print(f"type            : {device_type}")
    print(f"ipaddress       : {ipaddress}")
    print(f"port            : {port}")
    print(f"connection_type : {connection_type}")
    print(f"location        : {location}")
    print("|-------------------------|")
```

Output :

```
|-------------------------|
device_name     : R1
type            : router
ipaddress       : 10.15.1.1
port            : 22
connection_type : ssh
```

```
location              : SALE-A
|-----------------------|
|-----------------------|
device_name           : SW1
type                  : switch
ipaddress             : 10.15.2.1
port                  : 22
connection_type       : ssh
location              : SALE-A
|-----------------------|
```

ipaddress Module

The **ipaddress** module is a built-in Python library that provides robust tools to validate, manipulate, and analyze IP addresses and networks ideal for automating subnetting tasks, generating IP pools, or validating input in scripts.

Example 1: Analyze an IPv4 Network

This script prompts the user to enter a network in CIDR format, then displays key information like available hosts, subnet mask, and broadcast address.

```
from ipaddress import IPv4Network

ipaddress = input("Enter your network IP address (e.g. 192.168.1.0/24): ")

try:
    net = IPv4Network(ipaddress)

    print("Available addresses:", net.num_addresses)
    print("Network ID and mask:", net.network_address, net.netmask)
    print("Broadcast address   :", net.broadcast_address)
    print("IP version          :", net.version)
    print("First usable IP     :", list(net.hosts())[0])
    print("Last usable IP      :", list(net.hosts())[-1])
except Exception as e:
    print("Error:", e)
```

Example 2: Subnet a Large Network into Smaller /28 Subnets

In this example, we'll take a large network (e.g., 192.168.0.0/24) and subdivide it into smaller subnets (e.g., /28).

```
from ipaddress import IPv4Network

# Input a large network
network = input("Enter a network to subnet (e.g. 192.168.0.0/24): ")

try:
    parent_net = IPv4Network(network)
    new_prefix = 28  # Target subnet mask (/28 = 16 addresses per subnet)
```

```
    print(f"\nSubnetting {parent_net} into /{new_prefix} blocks:\n")
    subnets = list(parent_net.subnets(new_prefix=new_prefix))

    for subnet in subnets:
        print(f"Subnet: {subnet}")
        print(f"  Network ID     : {subnet.network_address}")
        print(f"  Broadcast      : {subnet.broadcast_address}")
        print(f"  First usable   : {list(subnet.hosts())[0]}")
        print(f"  Last usable    : {list(subnet.hosts())[-1]}")
        print(f"  Total addresses: {subnet.num_addresses}")
        print("-" * 40)
except Exception as e:
    print("Error:", e)
```

Output :

```
Subnetting 192.168.1.0/24 into /28 blocks:

Subnet: 192.168.1.0/28
  Network ID     : 192.168.1.0
  Broadcast      : 192.168.1.15
  First usable   : 192.168.1.1
  Last usable    : 192.168.1.14
  Total addresses: 16
...
```

netaddr Library

The netaddr library is a feature-rich Python toolkit for handling IP addresses, subnets, MAC addresses, and other networking data types. It's particularly useful in network automation for tasks like IP allocation, MAC parsing, and network validation.

To use netaddr, install it using:

```
pip install netaddr
```

Key Features of netaddr :

- **IP Addressing :** Parsing, validation, conversion, subnet analysis, and iteration.
- **Subnetting :** CIDR calculations, broadcast, netmask, supernetting/subnetting.
- **MAC Addressing :** Format normalization, vendor lookup, and EUI support.
- **Address Ranges :** Set operations (union, intersection), sorting, comparisons.

Example 1: IP Subnet Analysis

We'll replicate the IPv4 subnetting logic from earlier, this time using **netaddr.**

```
from netaddr import IPNetwork, AddrFormatError

ipaddress = input("Enter your Network IP address (e.g. 192.168.1.0/24): ")
```

```
try:
    net = IPNetwork(ipaddress)

    print("Available addresses:", net.size)
    print("Network ID and mask:", net.network, net.netmask)
    print("Broadcast:", net.broadcast)
    print("IP version:", net.version)
    print("First usable IP:", list(net)[1])  # Skipping network address
    print("Last usable IP :", list(net)[-1]) # Broadcast still included for reference

except AddrFormatError as e:
    print(f"Invalid IP address format: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

Output :

```
Enter your Network IP address (e.g. 192.168.1.0/24): 192.168.1.0/24
Available addresses: 256
Network ID and mask: 192.168.1.0 255.255.255.0
Broadcast: 192.168.1.255
IP version: 4
First usable IP: 192.168.1.1
Last usable IP : 192.168.1.255
```

Example 2: MAC Address Analysis and Vendor Lookup

Using EUI from netaddr, you can handle and format MAC addresses, as well as retrieve vendor details (if supported).

```
from netaddr import EUI

# Create a MAC address object
mac = EUI('00-1B-44-11-3A-B7')
# Print the MAC address in different formats
print(mac)
print(mac.words)
print(mac.oui)
#Get the vendor.
try:
    print(mac.oui.registration().org)
except:
    print("Vendor information not available")
```

Output :

```
00-1B-44-11-3A-B7
(0, 27, 68, 17, 58, 183)
00-1B-44
SanDisk Corporation
```

os Module

The os module is a built-in Python library that allows interaction with the operating system. It's especially useful in network automation for creating logs, managing directories for backups or outputs, and navigating the file system dynamically.

1. **File and Directory Management**

   These operations help automate the creation, navigation, and cleanup of folders used for configuration storage, logs, or output files.

Example 1: Creating a Directory

```
import os

# Create a directory named "network_automation"
os.mkdir("network_automation")
```

Example 2: Listing Files and Folders

```
import os

# List contents of the current working directory
files = os.listdir()
print(files)
```

Example 3: Removing a Directory

```
import os

# Remove an empty directory (must not contain files)
os.rmdir("network_automation")
```

2. **Path Manipulation**

   Path operations are useful when handling files across multiple directories or dynamically generating paths.

Example 1 : Get Current Working Directory

```
import os

current_dir = os.getcwd()
print("Current directory:", current_dir)
```

Example 2 : Change Directory

```
import os

# Navigate to the "network_automation" directory
os.chdir("network_automation")
```

Example 3 : Join Paths Properly

Use **os.path.join()** to avoid issues with slashes across different operating systems.

```
import os

file_path = os.path.join("network_automation", "data.txt")
print("File path:", file_path)
```

3. **Accessing Environment Variables**

   Environment variables are useful for user authentication, dynamic file paths, or storing credentials securely.

Example :

```
import os

# On Windows: Get the USERNAME environment variable
username = os.environ["USERNAME"]
print("Logged-in user:", username)
```

Tip: *Use* ***os.environ.get("VAR_NAME", "default_value")*** *to avoid errors if a variable doesn't exist.*

4. **Running System Commands with subprocess & os.system()**

   The subprocess module allows you to spawn new processes, connect to their input/output/error pipes, and retrieve results. It's a better and more flexible alternative to os.system().

Feature	os.system()	subprocess.run() (preferred)
Output Handling	Can't directly capture output	Can capture and manipulate output
Error Handling	Limited	Robust (with return codes, exceptions)
Security	Less safe (shell injection risk)	Safer when used with lists

Example 1: Using **subprocess** to Ping a Network Device

```
import subprocess

ip = input("Enter IP address to ping: ")

try:
    result = subprocess.run(["ping", "-n", "2", ip], capture_output=True, text=True)
   # For Linux/macOS, replace "-n" with "-c"

    print("Ping Output:\n", result.stdout)
except Exception as e:
    print("Error running ping command:", e)
```

Output :

```
Enter IP address to ping: 8.8.8.8
Ping Output:

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=49ms TTL=118
Reply from 8.8.8.8: bytes=32 time=50ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 49ms, Maximum = 50ms, Average = 49ms
```

Example 2: Using **os.system()** to Ping a Network Device

```
import os

ip = input("Enter the IP address to ping: ")

# Ping the IP address using a system call
response = os.system(f"ping -n 2 {ip}")  # For Windows
# For Linux/macOS, replace "-n" with "-c"

# Optional: Check response code (0 = success)
if response == 0:
    print(f"{ip} is reachable.")
else:
    print(f"{ip} is unreachable.")
```

Output :

```
Enter the IP address to ping: 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=57ms TTL=118
Reply from 8.8.8.8: bytes=32 time=48ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 48ms, Maximum = 57ms, Average = 52ms
8.8.8.8 is reachable.
```

PrettyTable Library

PrettyTable is a Python library that makes it easy to create clean, readable tables in the terminal. It's especially useful for presenting device inventory, interface status, IP allocations, and more in a clear format ideal for command-line tools or reports.

Install it using:

```
pip install prettytable
```

Example : Displaying Device Inventory

```
import prettytable

# Simulate network device data
devices = [
{'hostname': 'device1', 'ip_address': '10.0.0.1', 'model': 'Cisco IOSv15.2', 'status': 'Up'},
{'hostname': 'device2', 'ip_address': '192.168.1.1', 'model': 'Juniper EX4300', 'status':
'Down'},
{'hostname': 'device3', 'ip_address': '172.16.0.100', 'model': 'Arista DCS-7010', 'status':
'Up'}
]

# Create a PrettyTable object with column headers
table = prettytable.PrettyTable(['Hostname', 'IP Address', 'Model', 'Status'])
table.align = 'l'  # Left-align columns

# Add device data to the table
for device in devices:
    table.add_row([device['hostname'], device['ip_address'], device['model'],
device['status']])

# Print the formatted table
print(table)
```

Output :

```
+-------------+-----------------+---------------------+--------+
| Hostname | IP Address     | Model                | Status |
+-------------+-----------------+---------------------+--------+
| device1     | 10.0.0.1        | Cisco IOSv15.2   | Up     |
| device2     | 192.168.1.1     | Juniper EX4300  | Down |
| device3     | 172.16.0.100  | Arista DCS-7010 | Up     |
+-------------+-----------------+---------------------+--------+
```

Regular Expressions (re Module)

Regular expressions (RegEx) are powerful tools used to match, extract, and manipulate text patterns. In network automation, they are invaluable for parsing command outputs, logs, and configuration files.

Python provides the **re module** for working with regular expressions.

Basic RegEx Components

Element	Description
.	Matches **any character** except newline
^ / $	Start / end of string
[abc] / [a-z]	Matches any one character in the set or range
***, +, ?**	Match zero/more, one/more, or zero/one repetitions

{m,n}	Matches **m to n repetitions**
()	Grouping for **capturing subpatterns**

Common RegEx Functions

Function	Description
re.match()	Checks if the pattern matches **at the beginning** of a string
re.search()	Searches for the **first match anywhere** in the string
re.findall()	Returns a **list of all matches**
re.sub()	**Replaces** occurrences of a pattern with a string

Sample CLI Output: We'll reuse this output in the following examples:

```
CSR1000V#show interfaces gigabitEthernet 1
GigabitEthernet1 is up, line protocol is up
  Hardware is CSR vNIC, address is 0cf6.c711.0000 (bia 0cf6.c711.0000)
  Description: TO-HOME-WIFI-NETWORK
  Internet address is 192.168.1.50/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full Duplex, 1000Mbps, link type is auto, media type is Virtual
  output flow-control is unsupported, input flow-control is unsupported
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 00:00:11, output 00:02:05, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/375/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
  Output queue: 0/40 (size/max)
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
     51 packets input, 5392 bytes, 0 no buffer
     Received 0 broadcasts (0 IP multicasts)
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     0 watchdog, 0 multicast, 0 pause input
     1 packets output, 60 bytes, 0 underruns
     0 output errors, 0 collisions, 0 interface resets
     0 unknown protocol drops
     0 output buffer failures, 0 output buffers swapped out
```

Example 1 : ***re.match()*** – Checks Only the Start of the String

```
import re

show_interface_output = """CSR1000V#show interfaces gigabitEthernet 1"""

# Try matching the hostname at the very beginning of the output
match_result = re.match(r"\w+#", show_interface_output)
```

```
if match_result:
    print("Match found:", match_result.group())
else:
    print("No match found.")
```

Output :

```
Match found: CSR1000V#
```

Example 2 : *re.search()* – Scans for the First Match Anywhere

re.search() returns the first match of the pattern anywhere in the string.

```
import re

# The variable contains the sample CLI output shown above
show_interface_output = """..."""

search_result = re.search(r"GigabitEthernet\d+", show_interface_output)
if search_result:
    print("Interface found:", search_result.group())
```

Output :

```
Interface found: GigabitEthernet1
```

Example 3 : *re.findall()* – Returns All Matches in a List

re.findall() returns all non-overlapping matches as a list.

```
import re

# The variable contains the sample CLI output shown above
show_interface_output = """..."""

numbers = re.findall(r"\d+:\d+:\d+", show_interface_output)
print("timers:", numbers)
```

Output :

```
timers: ['04:00:00', '00:00:11', '00:02:05']
```

Example 4 : *re.sub()* – Substitutes Matching Text

re.sub() replaces all matches of the pattern with the replacement string.

```
import re

# The variable contains the sample CLI output shown above
show_interface_output = """..."""

masked_output = re.sub(r"[0-9a-f]{4}\.[0-9a-f]{4}\.[0-9a-f]{4}", "[MAC-HIDDEN]",
show_interface_output)
print(masked_output.splitlines()[2])  # Print only the MAC line for brevity
```

Output :

```
Hardware is CSR vNIC, address is [MAC-HIDDEN] (bia [MAC-HIDDEN])
```

Example 5 : Using re.search() with Groups

Use **()** to capture groups inside your pattern.

```
import re

# The variable contains the sample CLI output shown above
show_interface_output = """..."""

# Extract and separate the IP address and CIDR from the interface output
ip_pattern = re.search(r"Internet address is (\d{1,3}(?:\.\d{1,3}){3})/(\d{1,2})",
show_interface_output)
if ip_pattern:
    print("IP Address:", ip_pattern.group(1))
    print("CIDR Mask :", ip_pattern.group(2))
```

Output :

```
IP Address: 192.168.1.50
CIDR Mask : 24
```

Command-Line Arguments

Command-line arguments let users pass values to your script at runtime ideal for dynamic operations like supplying usernames, passwords, device IPs, or config files without editing the script.

Python offers two ways to handle command-line arguments:

1. **Using *sys.argv* (Basic Method) :**The sys module provides a list called **argv**, where:
    a. ***sys.argv[0]*** is the name of the script.
    b. ***sys.argv[1], sys.argv[2***], etc., are user-supplied arguments.

Example :

```
import sys

# Usage: python script.py <username> <password>
name_script = sys.argv[0]
username = sys.argv[1]
password = sys.argv[2]

print(f"name of the script  : {name_script}")
print(f"username            : {username}")
print(f"password            : {password}")
```

Command :

```
python script.py python_user Python123*
```

Output :

```
name of the script  : .\script.py
username            : python_user
password            : Python123*
```

> ***sys.argv*** *works well for simple scripts but lacks validation or help messages.*

2. **Using argparse (Recommended for Production) :** ***argparse*** is a more advanced and user-friendly way to handle command-line arguments. It automatically generates help messages and supports optional flags.

Example :

```
import argparse

parser = argparse.ArgumentParser(description="Network automation script")
parser.add_argument("-u", "--username", help="Username for device access")
parser.add_argument("-p", "--password", help="Password for device access")

args = parser.parse_args()

try:
    print(f"Username: {args.username}")
    print(f"Password: {args.password}")
except Exception as e:
    print("Error:", e)
```

Command :

```
python script.py -u python_user -p Python123*
```

Output :

```
Username: python_user
Password: Python_123*
```

python-dotenv Library

In network automation, you often deal with sensitive data like credentials, IP addresses, or tokens. Hardcoding these into your Python scripts is risky. Instead, you can use the python-dotenv library to store and access them securely using environment variables.

What Is python-dotenv?

python-dotenv allows you to load environment variables from a ***.env file*** into your Python script. It helps:

- Keep secrets out of source code
- Make your scripts more portable and secure
- Work seamlessly with tools like os.environ

Install it using:

```
pip install python-dotenv
```

Example :

1. **Create a .env File :** Create a file named **.env** in the root directory of your project with the following content:

```
USERNAME=python_user
PASSWORD=Python123*
DEVICE_IP=192.168.1.1
```

2. **Access Variables in Your Script :**

```
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()
 # Retrieve variables
username = os.getenv("USERNAME")
password = os.getenv("PASSWORD")
device_ip = os.getenv("DEVICE_IP")

print(f"Username: {username}")
print(f"Password: {password}")
print(f"Device IP: {device_ip}")
```

Output :

```
Username: python_user
Password: Python123*
Device IP: 192.168.1.1
```

Working with Sockets in Python

Sockets are endpoints of a communication link between two programs running on a network. They facilitate inter-process communication by establishing contact points for data exchange. Each socket has a specific address composed of an IP address and a port number. Sockets are crucial in client-server applications, where the server creates a socket, attaches it to a network port, and waits for client connections. There are two main types of sockets: datagram sockets, which provide connectionless communication, and stream sockets, which offer connection-oriented, sequenced data flow without record boundaries.

Python provides a built-in socket module to help work with sockets, here an example of TCPserver and TCPclient using socket:

Example 1: TCP Server

This example demonstrates a basic Python server that listens on port 12000 for incoming TCP connections, receives a message from a client, converts it to uppercase, and sends it back to the client.

```
from socket import *

serverport = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)

serverSocket.bind(('127.0.0.1', serverport))
serverSocket.listen(1)
print("the server is ready")

while True:
    connectionSock, addr = serverSocket.accept()
    message = connectionSock.recv(1024).decode()
    server_message = f"form the server: {message}"
    connectionSock.send(server_message.upper().encode())
    connectionSock.close()
```

Example 2: TCP Client

This example demonstrates a basic Python TCP client that establishes a connection to a server hosted locally at IP address 127.0.0.1 on port 12000.

```
from socket import *

serverip = "127.0.0.1"
serverport = 12000
clientsocket = socket(AF_INET, SOCK_STREAM)
clientsocket.connect((serverip,serverport))

messege = input("entre your message: ")
clientsocket.send(messege.encode())
modifidemessage = clientsocket.recv(1024)

print(f"{modifidemessage.decode()}")
clientsocket.close()
```

Output : TCP server

```
The server is ready
```

Output : TCP client

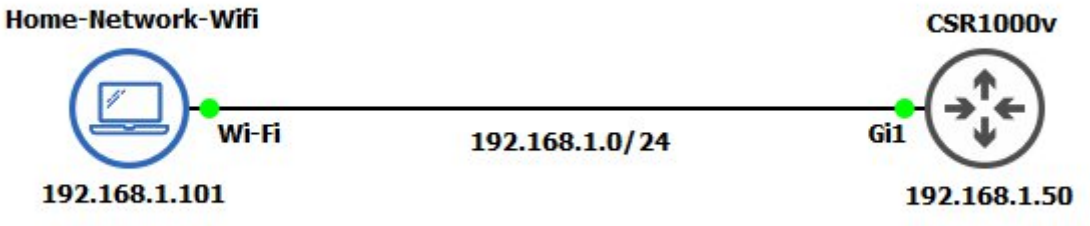
```
entre your message: hi tcp
FORM THE SERVER: HI TCP
```

Lab Preparation: Setting Up the CSR1000V Router in GNS3

Before diving into Netmiko and other network automation tools, you'll first need to set up a virtual lab using GNS3 and a Cisco CSR1000V router. This lab will serve as your environment for testing automation scripts and practicing device interaction.

Steps :

- you need to install GNS3
- you need CSR1000V image you can find Tutorial on youtube of how to setup the image.
- Example of the topology:



Router Config:

- router name : CSR1000V
- user: pyhton
- password: python123
- password for enable mode: python123
- SSH version 2 (version 2 is required), Telnet
- router ip address: 192.168.1.50

```
clock set 22:41:53 april 13 2018
config terminal
hostname CSR1000V
no ip domain lookup
ip domain name pyhton.local

crypto key generate rsa modulus 1024
ip ssh version 2

username python privilege 15 password python123
enable secret python123

line vty 0 4
logging synchronous
login local
transport input telnet ssh
exit
```

```
interface gi 1
ip address 192.168.1.50 255.255.255.0
description TO-HOME-WIFI-NETWORK
no shutdown
exit
netconf-yang
ip ftp source-interface gigabitEthernet 1
```

Telnet (telnetlib)

Telnet is a protocol that enables remote access to a device's command-line interface (CLI) over a TCP connection. While it's considered less secure than SSH, it's still commonly used in lab environments or legacy network devices.

Python's ***telnetlib*** module allows you to script Telnet sessions making it useful for automating tasks on routers, switches, or firewalls that support Telnet.

Example 1: Run a show Command and Save Output

This script connects to a CSR1000V router, runs show interface description, and saves the output to a text file.

```
import telnetlib
import getpass

HOST = "192.168.1.50"
username = input("Enter your username: ")
password = getpass.getpass()

try:
    connection = telnetlib.Telnet(HOST,timeout=5)

    connection.read_until(b"Username: ")
    connection.write(username.encode('ascii') + b"\n")
    if password:
        connection.read_until(b"Password: ")
        connection.write(password.encode('ascii') + b"\n")
    connection.read_until(b"#")
    connection.write(b"\n")
    connection.write(b"show interface description\n")
    connection.write(b"!end of command\n")
    output = connection.read_until(b"!end of command").decode('ascii')
    print(output)
    with open("show-interface-description.txt", "w", newline="") as show_output:
        show_output.write(str(output))

    connection.close()
except Exception as exp:
        print(f"Connection Error: {exp}")
```

Output :

```
Enter your username: python
Password:

CSR1000V#show interface description
Interface                  Status         Protocol Description
Gi1                        up             up       TO-HOME-WIFI-NETWORK
Gi2                        admin down     down
Gi3                        admin down     down
Gi4                        admin down     down
CSR1000V#!end of command
```

Example 2: Configure a Loopback Interface via Telnet

This script connects via Telnet and configures a loopback interface, then logs the session output.

```
import telnetlib
import getpass

HOST = "192.168.1.50"
username = input("Enter your username: ")
password = getpass.getpass()

try:
    connection = telnetlib.Telnet(HOST,timeout=5)

    connection.read_until(b"Username: ")
    connection.write(username.encode('ascii') + b"\n")
    if password:
        connection.read_until(b"Password: ")
        connection.write(password.encode('ascii') + b"\n")
    connection.read_until(b"#")
    connection.write(b"\n")
    connection.write(b"config terminal\n")
    connection.write(b"interface loopback 0\n")
    connection.write(b"ip address 10.10.10.10 255.255.255.255\n")
    connection.write(b"description Python-Script-Telnet\n")
    connection.write(b"exit\n")
    connection.write(b"exit\n")
    connection.write(b"!end of command\n")
    output = connection.read_until(b"!end of command").decode('ascii')
    print(output)
    with open("logs.txt", "w", newline="") as show_output:
        show_output.write(str(output))

    connection.close()
except Exception as exp:
        print(f"Connection Error: {exp}")
```

Output :

```
Enter your username: python
Password:
```

```
CSR1000V#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CSR1000V(config)#interface loopback 0
CSR1000V(config-if)#ip address 10.10.10.10 255.255.255.255
CSR1000V(config-if)#description Python-Script-Telnet
CSR1000V(config-if)#exit
CSR1000V(config)#exit
CSR1000V#!end of command
```

SSH and Paramiko

SSH (Secure Shell) is a protocol that enables encrypted remote login and command execution over a network. Unlike Telnet, SSH ensures confidentiality, integrity, and authentication making it the preferred method for managing network devices securely.

Python provides paramiko, a powerful library to script SSH sessions for automation tasks.

Install Paramiko using:

```
pip install paramiko
```

SSH Protocol Architecture

Sub-Protocol	Description
SSH Transport Layer	Manages encryption, integrity, and key exchange
SSH User Authentication	Verifies users via passwords or public keys
SSH Connection Protocol	Enables multiple logical channels over a single secure session

Example 1: Run a show Command via SSH

This script connects to a Cisco CSR1000v router via SSH, executes show interface description, and saves the output to a file.

```
import time
import paramiko
import getpass

HOST = "192.168.1.50"
username = input("Enter your username: ")
password = getpass.getpass("Enter your password: ")
port = 22

try:
    # Create an SSH client and connect to the device
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(
```

```
            hostname=HOST, username=username, password=password,
                port=port, look_for_keys=False, allow_agent=False
            )

    # Start an interactive shell session
    shell = ssh.invoke_shell()
    # Send commands and receive output
    command = "show interface description"
    shell.send(command + '\n')
    time.sleep(1)  # Add a delay to allow time for command execution
    output = shell.recv(99999).decode('utf-8')
    print(output)
    with open("show-interface-description.txt", "w", newline="") as show_output:
        show_output.write(str(output))
    # Close the SSH connection
    ssh.close()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
Enter your username: python
Enter your password:

Interface                      Status         Protocol Description
Gi1                            up             up       TO-HOME-WIFI-NETWORK
Gi2                            admin down     down
Gi3                            admin down     down
Gi4                            admin down     down
Lo0                            up             up       Python-Script-Telnet
```

Example 2: Configure a Loopback Interface via SSH

This script connects to a Cisco CSR1000v router via SSH and configures a loopback interface.

```
import paramiko
import getpass
import time

HOST = "192.168.1.50"
username = input("Enter your username: ")
password = getpass.getpass("Enter your password: ")
port = 22

try:
    # List of commands to execute
    commands = [
        "terminal length 0",
        'config terminal',
```

```
        'interface loopback 1',
        'ip address 10.10.10.20 255.255.255.255',
        'description Python-Script-SSH',
        'exit',
        'exit'
    ]

    # Create an SSH client and connect to the device
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(
                hostname=HOST, username=username, password=password,
                    port=port, look_for_keys=False, allow_agent=False
                )

    # Start an interactive shell session
    shell = ssh.invoke_shell()

    # Send commands and receive output
    for command in commands:
        shell.send(command + '\n')
        time.sleep(1)  # Add a delay to allow time for command execution
        output = shell.recv(99999).decode('utf-8')
        with open("logs.txt", "a", newline="") as show_output:
         show_output.write(str(output))

    # Close the SSH connection
    ssh.close()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
Enter your username: python
Enter your password:

CSR1000V#terminal length 0
CSR1000V#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CSR1000V(config)#interface loopback 1
CSR1000V(config-if)#ip address 10.10.10.20 255.255.255.255
CSR1000V(config-if)#description Python-Script-SSH
CSR1000V(config-if)#exit
CSR1000V(config)#exit
CSR1000V#
```

Netmiko

Netmiko is a Python library built on top of Paramiko, designed specifically for automating interactions with network devices over SSH (and optionally Telnet). It

abstracts much of the complexity involved in connecting to devices, sending commands, and handling responses.

Install Netmiko using:

```
pip install netmiko
```

Why Use Netmiko?

Feature	Description
Vendor Agnostic	list of device supported.
Simplified API	Makes connecting and sending commands easier than raw Paramiko
Secure	Uses SSH for encrypted sessions (or Telnet for legacy devices if required)
Built-in Handling	Manages device prompts, command execution, and config modes automatically

Example 1: Execute a show Command

This example connects to a Cisco CSR1000V router and runs show interface description, then prints the output.

```
from netmiko import ConnectHandler

# Device information
device_type = 'cisco_ios_telnet' # cisco_ios_telnet for telnet, cisco_ios of ssh
ip_address = '192.168.1.50'
username = 'python'
password = 'python123'
try:
    # Connect to the device
    connection = ConnectHandler(device_type=device_type, ip=ip_address,
                                username=username, password=password)

    # Send a command and capture the output
    show_output = connection.send_command('show interface description')

    # Print the output
    print(show_output)

    # Close the connection
    connection.disconnect()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
Interface                  Status         Protocol Description
Gi1                        up             up       TO-HOME-WIFI-NETWORK
Gi2                        admin down     down
```

```
Gi3                    admin down     down
Gi4                    admin down     down
Lo0                    up             up       Python-Script-Telnet
Lo1                    up             up       Python-Script-SSH
```

Example 2: Push Configuration Commands

This script uses Netmiko to configure a loopback interface on the router.

```
from netmiko import ConnectHandler

# Device information
device_type = 'cisco_ios_telnet' # cisco_ios_telnet for telnet, cisco_ios of ssh
ip_address = '192.168.1.50'
username = 'python'
password = 'python123'
try:
    # Connect to the device
    connection = ConnectHandler(device_type=device_type, ip=ip_address,
                                username=username, password=password)
    commands = [
        'interface loopback 2',
        'ip address 10.10.10.30 255.255.255.255',
        'description Python-Script-Netmiko',
        'end'
    ]
    # Send a command and capture the output
    output = connection.send_config_set(commands)

    # Print the output
    print(output)

    # Close the connection
    connection.disconnect()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CSR1000V(config)#interface loopback 2
CSR1000V(config-if)#ip address 10.10.10.30 255.255.255.255
CSR1000V(config-if)#description Python-Script-Netmiko
CSR1000V(config-if)#end
CSR1000V#
```

Example 3: Handle Prompt

Some commands (like saving the config) require interaction. Netmiko's ***send_multiline()*** handles this smoothly.

```
from netmiko import ConnectHandler
```

```
# Device information
device_type = 'cisco_ios_telnet' # cisco_ios_telnet for telnet, cisco_ios of ssh
ip_address = '192.168.1.50'
username = 'python'
password = 'python123'
try:
    # Connect to the device
    connection = ConnectHandler(device_type=device_type, ip=ip_address,
                            username=username, password=password)
    commands = [
        ["copy running-config startup-config",r"\[startup-config\]\?"],
        ["\n",""]
    ]
    # Send a command and capture the output
    output = connection.send_multiline(commands)

    # Print the output
    print(output)

    # Close the connection
    connection.disconnect()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
CSR1000V#
```

Screen Scraping

Screen scraping is a technique used to extract data from network devices, especially those that do not offer APIs or structured data interfaces. It's particularly useful for working with legacy systems or CLI-only environments.

How It Works:

- **Connection:** A Python script (often using Telnet, SSH, or Netmiko) connects to a device.
- **Command Execution:** It sends CLI commands like show interfaces or show version.
- **Output Capture:** The raw terminal output is captured as plain text.
- **Parsing:** The script then uses tools like regular expressions, TextFSM, TTP to extract meaningful values.
- Storage/Conversion: Parsed data is stored in logs or structured formats for reporting or analysis.

Parsing CLI Output with TextFSM

TextFSM is a Python module designed to parse structured text like the output from network device CLIs. It overcomes the limitations of traditional screen scraping by offering a template-based, reliable, and reusable way to extract data from command outputs.

Example: Parse **show interfaces gigabitEthernet 1** Output with TextFSM

This example demonstrates how to use a TextFSM template to extract data from a router **show interfaces gigabitEthernet 1** command.

1. **Create a Template File :** Filename: ***template.textfsm***

```
Value router_name (\S+)
Value interface_name (GigabitEthernet\d+)
Value description (\S+)
Value ipaddress (([0-9]{1,3}\.){3}[0-9]{1,3}\/[0-9]{2})
Value macaddress ([0-9a-f]{4}\.[0-9a-f]{4}\.[0-9a-f]{4})
Value bandwidth (\d+ \w+\/\w+)

Start
  ^${router_name}#
  ^${interface_name}
  ^  Description: ${description}
  ^  Internet address is ${ipaddress}
  ^  Hardware is CSR vNIC, address is ${macaddress}
  ^  MTU \d+ bytes, BW ${bandwidth} -> Record
```

2. **Python Script to Parse the Output**

```
import textfsm
import prettytable


output = """
CSR1000V#show interfaces gigabitEthernet 1
GigabitEthernet1 is up, line protocol is up
  Hardware is CSR vNIC, address is 0cf6.c711.0000 (bia 0cf6.c711.0000)
  Description: TO-HOME-WIFI
  Internet address is 192.168.1.50/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full Duplex, 1000Mbps, link type is auto, media type is Virtual
  output flow-control is unsupported, input flow-control is unsupported
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 00:00:11, output 00:02:05, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/375/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
```

```
  Output queue: 0/40 (size/max)
"""
template = "template.textfsm"


# Open the TextFSM template file
with open(template) as f:

    # Load the template using TextFSM
    re_table = textfsm.TextFSM(f)

    # Get the variable names defined in the template (e.g., interface_name, etc.)
    header = re_table.header

    # Parse the CLI output using the template
    result = re_table.ParseText(output)

    # Initialize an empty list to store parsed data as dictionaries
    interface_info = []

    # Convert each row (list of values) into a dictionary with header as keys
    for row in result:
        interface_info.append(dict(zip(header, row)))

# Create a PrettyTable object with column headers
table = prettytable.PrettyTable(['router_name', 'ipaddress', 'interface_name',
                                 'description','macaddress','bandwidth'])
table.align = 'l'  # Left-align columns

# Add device data to the table
for device in interface_info:
    table.add_row([device['router_name'], device['ipaddress'], device['interface_name'],
                   device['description'], device['macaddress'], device['bandwidth']])

print(table)
```

Output :

```
+-----------------+---------------------+---------------------+---------------------+----------------------+-------------------------+
| router_name | ipaddress         | interface_name | description    | macaddress           | bandwidth           |
+-----------------+---------------------+---------------------+---------------------+----------------------+-------------------------+
| CSR1000V    | 192.168.1.50/24 | GigabitEthernet1 | TO-HOME-WIFI | 0cf6.c711.0000 | 1000000 Kbit/sec |
+-----------------+---------------------+---------------------+---------------------+----------------------+-------------------------+
```

Parsing CLI Output with TTP (Template Text Parser)

TTP is a Python library designed to extract structured data from unstructured text output, like that from network devices. It provides a template-driven parsing engine with built-in support for macros, RegEx, filters, and modifiers, allowing for clean, readable, and reusable templates.

Install it using:

```
pip install ttp
```

Example: Parse **show interfaces gigabitEthernet 1** Output with TTP

This example demonstrates how to parse **show interfaces gigabitEthernet 1** output from a Cisco CSR1000V router using a TTP template.

```
from ttp import ttp
from pprint import pprint
import json


template = r"""
<vars>
interface = "GigabitEthernet\\d+"
ip = "([0-9]{1,3}\\.){3}[0-9]{1,3}\\/[0-9]{2}"
mac = "[0-9a-f]{4}\\.[0-9a-f]{4}\\.[0-9a-f]{4}"
bw = "\\d+ \\w+\\/\\w+"
</vars>


<group name="interfaces">
{{router_name}}#{{ignore(ORPHRASE)}}
{{interface_name | re(interface)}} {{ignore(ORPHRASE)}}
  Description: {{description | ORPHRASE}}
  Internet address is {{ipaddress | re(ip)}}
  Hardware is CSR vNIC, address is {{macaddress | re(mac)}} {{ignore(ORPHRASE)}}
  {{ignore(ORPHRASE)}} BW {{bandwidth | re(bw)}} {{ignore(ORPHRASE)}}
</group>
"""


output = """
CSR1000V#show interfaces gigabitEthernet 1
GigabitEthernet1 is up, line protocol is up
  Hardware is CSR vNIC, address is 0cf6.c711.0000 (bia 0cf6.c711.0000)
  Description: TO-HOME-WIFI-NETWORK
  Internet address is 192.168.1.50/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full Duplex, 1000Mbps, link type is auto, media type is Virtual
  output flow-control is unsupported, input flow-control is unsupported
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 00:00:11, output 00:02:05, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/375/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
  Output queue: 0/40 (size/max)
"""

# create parser object and parse data using template:
parser = ttp(data=output, template=template)
parser.parse()

# Get the first JSON result as a string
```

```
result_json = parser.result(format='json')[0]
# Convert to Python dictionary
result_dict = json.loads(result_json)

pprint(result_dict)
```

Output :

```
[{'interfaces': {'description': 'TO-HOME-WIFI-NETWORK',
                 'interface_name': 'GigabitEthernet1',
                 'ipaddress': '192.168.1.50/24',
                 'macaddress': '0cf6.c711.0000',
                 'router_name': 'CSR1000V'}}]
```

When to Use TTP vs. TextFSM

Feature	TextFSM	TTP
Template Format	RegEx-based, strict syntax	Simplified syntax + built-in filters
Pre/Post Processing	Requires external logic	Built-in modifiers (no post needed)
Output Formats	List of dicts (manual export)	JSON, YAML, CSV, Table
Complexity Handling	Better for simple blocks	Better for complex multiline parsing

Netmiko with TextFSM and TTP

When automating CLI data collection with Netmiko, you often need to convert raw command output into structured, machine-readable formats like JSON or Python dictionaries. Netmiko supports two powerful parsing methods:

- TextFSM
- TTP (Template Text Parser)

Example 1: Netmiko with TextFSM

TextFSM is natively supported by Netmiko via the **use_textfsm=True** argument. You need a custom .textfsm template file to define the parsing rules.

Step 1: Create the Template

Filename: show_interface.textfsm

```
Value interface_name (GigabitEthernet\d+)
Value description (\S+)
Value ipaddress (([0-9]{1,3}\.){3}[0-9]{1,3}\/[0-9]{2})
Value macaddress ([0-9a-f]{4}\.[0-9a-f]{4}\.[0-9a-f]{4})
Value bandwidth (\d+ \w+\/\w+)

Start
  ^${interface_name}
  ^  Description: ${description}
  ^  Internet address is ${ipaddress}
  ^  Hardware is CSR vNIC, address is ${macaddress}
  ^  MTU \d+ bytes, BW ${bandwidth} -> Record
```

Step 2: Python Script

```
from netmiko import ConnectHandler
from pprint import pprint

# Device information
device_type = 'cisco_ios_telnet' # cisco_ios_telnet for telnet, cisco_ios of ssh
ip_address = '192.168.1.50'
username = 'python'
password = 'python123'
try:
    # Connect to the device
    connection = ConnectHandler(device_type=device_type, ip=ip_address,
                            username=username, password=password)

    # Send a command and capture the output
    output = connection.send_command("show interfaces gigabitEthernet 1",
                                use_textfsm=True, textfsm_template="./show_interface.textfsm")

    # Print the output
    pprint(output)

    # Close the connection
    connection.disconnect()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
[{'bandwidth': '1000000 Kbit/sec',
  'description': 'TO-HOME-WIFI-NETWORK',
  'interface_name': 'GigabitEthernet1',
  'ipaddress': '192.168.1.50/24',
  'macaddress': '0cf6.c711.0000'}]
```

Example 2: Netmiko with TTP

Netmiko also supports TTP parsing using **use_ttp=True**. TTP templates are more flexible, especially for multiline patterns or nested groups.

Step 1: Create the TTP Template

Filename: **show_interface.ttp**

```
<vars>
interface = "GigabitEthernet\d+"
ip = "([0-9]{1,3}\.){3}[0-9]{1,3}\/[0-9]{2}"
mac = "[0-9a-f]{4}\.[0-9a-f]{4}\.[0-9a-f]{4}"
bw = "\d+ \w+\/\w+"
</vars>

<group name="interfaces">
{{interface_name | re(interface)}} {{ignore(ORPHRASE)}}
  Description: {{description | ORPHRASE}}
  Internet address is {{ipaddress | re(ip)}}
  Hardware is CSR vNIC, address is {{macaddress | re(mac)}} {{ignore(ORPHRASE)}}
  {{ignore(ORPHRASE)}} BW {{bandwidth | re(bw)}} {{ignore(ORPHRASE)}}
```

```
</group>
```

Step 2: Python Script

```
from netmiko import ConnectHandler
from pprint import pprint

# Device information
device_type = 'cisco_ios_telnet' # cisco_ios_telnet for telnet, cisco_ios of ssh
ip_address = '192.168.1.50'
username = 'python'
password = 'python123'

try:
    # Connect to the device
    connection = ConnectHandler(device_type=device_type, ip=ip_address,
                                username=username, password=password)

    # Send a command and capture the output
    output = connection.send_command("show interfaces gigabitEthernet 1",
                                     use_ttp=True, ttp_template="./show_interface.ttp")

    # Print the output
    pprint(output)

    # Close the connection
    connection.disconnect()
except Exception as e:
    print(f"Error:\n {e}")
```

Output :

```
[[{'interfaces': {'description': 'TO-HOME-WIFI-NETWORK',
                  'interface_name': 'GigabitEthernet1',
                  'ipaddress': '192.168.1.50/24',
                  'macaddress': '0cf6.c711.0000'}}]]
```

Jinja

Jinja (commonly referred to as Jinja2) is a powerful templating engine widely used in Python-based network automation. It excels at generating structured text outputs such as device configurations by combining static content with dynamic data.

Install it using:

```
pip install Jinja2
```

Jinja Features :

- **Variables:** Use double curly braces (**{{ }}**) to embed dynamic placeholders within templates. These placeholders are replaced with actual values during the rendering process such as interface names, IP addresses, or VLAN IDs.

- **Conditional Statements:** Jinja supports logic through **if, elif, and else statements**. This allows you to include or exclude specific configuration blocks based on conditional checks.
- **Loops:** For repetitive tasks like generating multiple VLANs or interfaces, Jinja's for loops let you iterate through data structures to produce configurations efficiently.
- **Filters:** Jinja includes filters that help you modify data within templates. These are useful for formatting strings, changing data types, or applying transformations to lists and values.

Example: Automating Interface Configuration with Jinja

In this example, we'll generate interface configuration for a router using a Jinja template.

- Create a folder named templates to store your Jinja templates.
- Inside the templates folder, create a file named interface.j2 with the following content:

interface.j2 Template:

```
interface {{ interface_info.interface_name }}
 description {{ interface_info.interface_description }}
 ip address {{ interface_info.ip_address }} {{ interface_info.subnet_mask }}
{% if interface_info.interface_state == true %}
 no shutdown
{% else %}
 shutdown
{% endif %}
```

Python Script :

```
import jinja2

# Load template and data
template_loader = jinja2.FileSystemLoader("templates")
template_env = jinja2.Environment(loader=template_loader)
template = template_env.get_template("interface.j2")

interface_info = {
    "interface_name": "GigabitEthernet3",
    "interface_description": "jinja-interface-config",
    "ip_address": "10.10.1.1",
    "subnet_mask": "255.255.255.0",
    "interface_state": True
}


interface_config = template.render(interface_info=interface_info)

print(interface_config)
```

Output :

```
interface GigabitEthernet3
 description jinja-interface-config
 ip address 10.10.1.1 255.255.255.0
 no shutdown
```

NETCONF and YANG

NETCONF (Network Configuration Protocol) is a protocol used to remotely manage, configure, and monitor network devices. It relies on a secure, client-server model—typically over SSH and exchanges data using XML and RPC (Remote Procedure Call).

Key Features of NETCONF:

- **Remote Access:** Manage devices without physical access.
- **Structured Communication:** Uses XML for human-readable configs and RPCs for commands.
- **Secure:** Runs over SSH for encrypted sessions.
- **Datastores:**
    - running: Active config
    - candidate: Staging config
    - startup: Boot config

Common NETCONF Operations:

- get-config: Retrieve configuration
- edit-config: Modify settings
- copy-config: Copy between datastores
- delete-config: Remove config data
- get: Fetch config + operational state

YANG is a language used to define the structure and types of configuration data NETCONF works with. It models things like interfaces, routing, and ACLs using a clear hierarchy of containers, lists, and leaves.

Why YANG Matters:

- **Standardized:** Ensures consistency across vendors
- **Machine-Readable:** Enables automation and validation
- **Extensible:** Can be customized (augmented or deviated)

Sources of YANG Models:

- **IETF:** Industry standards
- **OpenConfig:** Vendor-neutral models
- **Vendors:** Cisco, Juniper, etc.

ncclient: Python Library for NETCONF

ncclient is a Python library used to interact with network devices over the NETCONF protocol. It allows you to retrieve and apply configurations using structured XML data.

install it using:

```
pip install ncclient
```

Example 1: Retrieve and Save Router Configuration

This script connects to a NETCONF-enabled device and saves the running configuration to an XML file.

```
from ncclient import manager

m = manager.connect(
    host="192.168.1.50",
    port=830,
    username="python",
    password="python123",
    hostkey_verify=False,
    device_params={'name': "csr"},
    timeout=700
    )

reply = m.get_config(source='running').data_xml

with open("csr_config.xml", "w") as f:
    f.write(reply)

m.close_session()
```

Example 2: Configure an Interface

This example configures GigabitEthernet3 with a description and IP address using NETCONF.

```
from ncclient import manager

try:
    m = manager.connect(
        host="192.168.1.50",
        port=830,
        username="python",
        password="python123",
        hostkey_verify=False,
        device_params={'name': "csr"},
        timeout=700
        )

    template = """
    <config>
        <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
            <interface>
                <name>GigabitEthernet3</name>
```

```
            <description>NETCONF-EDITE</description>
            <enabled>true</enabled>
            <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip" operation="replace">
              <address>
                <ip>1.1.1.1</ip>
                <netmask>255.255.255.0</netmask>
              </address>
            </ipv4>
          </interface>
        </interfaces>
    </config>
    """
    result = m.edit_config(target='running', config=template)

    if "<ok/>" in result.xml:
        print("OK")

except Exception as e:
    print(f"Error Happend:\n{e}")

m.close_session()
```

Example 3: Configure OSPF Routing

This example configures OSPF on a Cisco router using a NETCONF XML payload.

```
from ncclient import manager

try:
    m = manager.connect(
        host="192.168.1.50",
        port=830,
        username="python",
        password="python123",
        hostkey_verify=False,
        device_params={'name': "csr"},
        timeout=700
        )

    template = """
<config>
<native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <router>
        <ospf xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-ospf" operation="replace">
            <id>1</id>
            <router-id>1.1.1.1</router-id>
            <network>
                <ip>192.168.1.50</ip>
                <mask>0.0.0.255</mask>
                <area>0</area>
            </network>
        </ospf>
    </router>
</native>
</config>
    """
    result = m.edit_config(target='running', config=template)
```

```
    if "<ok/>" in result.xml:
        print("OK")

except Exception as e:
    print(f"Error Happend:\n{e}")

m.close_session()
```

Logging

Logging is essential in network automation for tracking events, capturing errors, and auditing actions performed on network devices. Unlike print() statements, the logging module allows you to manage output levels, save logs to files, and maintain structured records.

Example :

```
import logging

# Basic configuration
logging.basicConfig(
    filename='network_automation.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# Sample log entries
logging.info("Script started")
logging.warning("Device returned a non-critical warning")
logging.error("Failed to connect to router")
```

Pandas for Network Data Analysis

Pandas is a powerful Python library for working with structured data—think of it like an Excel spreadsheet or SQL table, but with Python-level control. It's especially useful in network automation for analyzing logs, interface data, configurations, and performance metrics.

Key Pandas Concepts

- **Series:** One-dimensional labeled array (like a single column).
- **DataFrame:** Two-dimensional labeled structure (like a spreadsheet or SQL table).

Example Dataset :

Sample interface data (CSV format):

```
RouterName,Interface,bandwidth kilobit/s,description
SWITCH-B1,Ethernet0/0/0,100000,TO-PC1-B1
SWITCH-B1,Ethernet0/0/1,100000,TO-PC2-B1
SWITCH-B1,Ethernet0/0/2,100000,TO-PC3-B1
```

```
SWITCH-B1,Ethernet0/0/3,100000,--
SWITCH-B1,Ethernet0/0/4,100000,--
SWITCH-B1,GigabitEthernet0/0/0,1000000,TO-SW2
SWITCH-B1,GigabitEthernet0/0/1,1000000,TO-SERVER-B1
SWITCH-B1,GigabitEthernet0/0/2,1000000,TO-INTERNET
SWITCH-B1,GigabitEthernet0/0/3,1000000,--
SWITCH-B2,Ethernet0/0/0,100000,TO-PC1-B2
SWITCH-B2,Ethernet0/0/1,100000,TO-PC2-B2
SWITCH-B2,Ethernet0/0/2,100000,--
SWITCH-B2,Ethernet0/0/3,100000,--
SWITCH-B2,Ethernet0/0/4,100000,--
SWITCH-B2,GigabitEthernet0/0/0,1000000,TO-SW1
SWITCH-B2,GigabitEthernet0/0/1,1000000,TO-SERVER-B2
SWITCH-B2,GigabitEthernet0/0/2,1000000,TO-INTERNET
SWITCH-B2,GigabitEthernet0/0/3,1000000,--
```

1. **Reading CSV Data**

```
import pandas as pd
# Read the CSV file
interfaces_df = pd.read_csv('interfaces.csv')
# Display the first few rows
print("First 5 rows of the dataset:")
print(interfaces_df.head())
```

2. **Data Selection and Filtering**

```
import pandas as pd

# Read the CSV file
interfaces_df = pd.read_csv('interfaces.csv')
# Select specific columns
router_interface_df = interfaces_df[['RouterName', 'Interface']]
print("\nRouter and Interface columns only:")
print(router_interface_df.head())

# Select high-speed interfaces
high_speed = interfaces_df[interfaces_df['bandwidth kilobit/s'] > 500000]
print("\nHigh-speed interfaces (>500 Mbps):")
print(high_speed)

# Filter data for a specific switch
switch_b1_df = interfaces_df[interfaces_df['RouterName'] == 'SWITCH-B1']
print("\nInterfaces on SWITCH-B1:")
print(switch_b1_df)
```

3. **Grouping and Aggregation**

```
import pandas as pd

# Read the CSV file
interfaces_df = pd.read_csv('interfaces.csv')
# Count interfaces by router
interfaces_by_router = interfaces_df.groupby('RouterName').size()
print("\nNumber of interfaces per router:")
print(interfaces_by_router)
```

Threading: Speeding Up Network Automation

When working with multiple network devices, running tasks sequentially can become slow and inefficient. Threading allows you to execute tasks in parallel, significantly improving performance especially when commands involve delays (e.g., SSH, Telnet, or API responses).

Python's **concurrent.futures.ThreadPoolExecutor** makes threading easy to implement and manage.

Example: Gather Hostnames from Multiple Routers

Let's assume you're connecting to a list of devices using Netmiko, and you want to run show interfaces on each one.

```
from netmiko import ConnectHandler
from concurrent.futures import ThreadPoolExecutor, as_completed


devices = [
    {"device_type": "cisco_ios", "ip": "192.168.1.50", "username": "python", "password":
"python123"},
    {"device_type": "cisco_ios", "ip": "192.168.1.2", "username": "python", "password":
"python123"},
    {"device_type": "cisco_ios", "ip": "192.168.1.3", "username": "python", "password":
"python123"},
]

def get_interfaces(device):
    try:
        connection = ConnectHandler(**device)
        output = connection.send_command("show interfaces")
        connection.disconnect()
        return f"{device['ip']} ➜ {output}"
    except Exception as e:
        return f"{device['ip']} ➜ ERROR: {e}"


# Set the number of threads (based on number of devices or system limits)
max_threads = 5

with ThreadPoolExecutor(max_workers=max_threads) as executor:
    futures = [executor.submit(get_interfaces, device) for device in devices]
    for future in as_completed(futures):
        print(future.result())
```

Config Diff: Comparing Network Configurations

In network automation, it's essential to detect changes between configurations. This can help identify:

- Unintended modifications
- Configuration drift
- Manual changes that deviate from automation

Python provides several tools to compare text-based configuration files, and the built-in **difflib** module is one of the easiest and most effective for this task.

Example: Compare Interface Config

```
import difflib

interface_config1 = """
interface GigabitEthernet3
 description jinja-interface-config
 ip address 10.10.1.1 255.255.255.0
 no shutdown
"""

interface_config2 = """
interface GigabitEthernet3
 description jinja-interface-config
 ip address 192.168.1.1 255.255.255.0
 shutdown
"""

# Convert multiline strings into lists of lines
config1_lines = interface_config1.strip().splitlines()
config2_lines = interface_config2.strip().splitlines()

# Create a diff
diff = difflib.unified_diff(
    config1_lines,
    config2_lines,
    fromfile='interface_config1',
    tofile='interface_config2',
    lineterm=''
)

# Print the differences
print('\n'.join(diff))
```

Output:

```
--- interface_config1
+++ interface_config2
@@ -1,4 +1,4 @@
 interface GigabitEthernet3
   description jinja-interface-config
- ip address 10.10.1.1 255.255.255.0
- no shutdown
+ ip address 192.168.1.1 255.255.255.0
+ shutdown
```

Conclusion

You've just taken a solid step into the world of network automation with Python. From scripting basics to interacting with network devices using tools like Netmiko, Paramiko,

TextFSM, TTP, and ncclient, you now have a practical toolkit for automating real-world tasks.

You've learned how to:

- Connect to routers and switches
- Automate configurations and collect data
- Parse and validate outputs
- Detect changes and manage configs at scale

This isn't the end it's your launchpad.

Continue practicing, expand into tools like RESTCONF, Nornir, or Ansible, and never stop improving your automation workflows.

Automation isn't about replacing engineers it's about empowering them. By automating the repetitive, you free yourself to focus on the strategic: improving security, scalability, and performance across your network.

Keep practicing, keep learning, and keep scripting.

The network won't automate itself but now you can.

Lahcen Khouchane.